

Lekce 16: Kurzory II - Editace tabulek

Python pro GIS - Update a Insert Cursor

Vojtěch Barták, FŽP ČZU Praha

2025-12-03

Table of contents

1	Úvod	3
1.1	Data	3
2	Přepsání hodnot v řádku pomocí Update kurzoru	3
2.1	Editace v cyklu	4
2.2	Úlohy	6
3	Smazání řádku	6
3.1	Smazání více záznamů v cyklu	7
3.2	Úlohy	8
4	Přidání nového řádku pomocí Insert kurzoru	8
4.1	Vytvoření nové tabulky a její naplnění daty	9
4.1.1	Verze s daty v kódu	10
4.1.2	Import dat z CSV souboru	11
4.2	Úlohy	13
5	Cheatsheet	13
5.1	Základy Update Cursor	13
5.1.1	Vytvoření Update kurzoru	13
5.1.2	Aktualizace řádku	14
5.1.3	Podmíněná aktualizace	14
5.2	Mazání řádků	15
5.2.1	Smazání konkrétního řádku	15
5.2.2	Smazání více řádků	15
5.2.3	Uložení před smazáním	15
5.3	Insert Cursor	16
5.3.1	Vytvoření Insert kurzoru	16

5.3.2	Vložení jednoho řádku	16
5.3.3	Vložení více řádků	16
5.3.4	Import z CSV	17
5.4	Práce s geometrií	17
5.4.1	Kopírování prvku včetně geometrie	17
5.4.2	Vytvoření bodu na souřadnicích	17
5.5	Vytváření tabulek	18
5.5.1	Vytvoření prázdné tabulky	18
5.5.2	Vytvoření a naplnění tabulky	18
5.6	Kombinace kurzorů	19
5.6.1	Kopírování s úpravou	19
5.6.2	Filtrované kopírování	19
5.6.3	Agregace dat	19
5.7	Časté vzory použití	20
5.7.1	Přidání vypočítaného pole	20
5.7.2	Kategorizace hodnot	20
5.7.3	Normalizace textu	21
5.8	Zpracování chyb	21
5.8.1	Try-except s kurzory	21
5.8.2	Kontrola existence pole	21
5.8.3	Ošetření None hodnot	22
5.9	Tipy pro výkon	22
5.9.1	Použití WHERE klauzule	22
5.9.2	Otevření pouze potřebných polí	22
5.9.3	Batch operace	23

1 Úvod

V předchozí lekci jsme se naučili číst data z atributových tabulek pomocí `SearchCursor`. V této lekci rozšíříme naše znalosti o další dva typy kurzorů, které umožňují data v tabulkách **editovat a přidávat nové záznamy**.

Po absolvování lekce budete umět:

- Měnit hodnoty v existujících řádcích pomocí `UpdateCursor`
- Mazat záznamy z tabulky
- Přidávat nové řádky pomocí `InsertCursor`
- Vytvářet nové tabulky a plnit je daty
- Importovat data z CSV souborů do GIS tabulek

 Pozor na nevratné změny!

Na rozdíl od `SearchCursor`, který data pouze čte, kurzory `UpdateCursor` a `InsertCursor` provádějí **trvalé změny** v datech. Před použitím těchto kurzorů si vždy:

- Zálohujte původní data
- Otestujte skript na kopii dat
- Používejte `arcpy.env.overwriteOutput = True` s rozmyslem

1.1 Data

V lekci budeme pracovat s následujícími daty:

- Data `Kraje_sidla` (stejná, jaké jsme používali v minulé lekci), obsahující bodový dataset vybraných obcí ČR `Sidla_body.shp` a polygonový dataset krajů ČR `Kraje.shp`.
- CSV tabulka `EU_OZE_Emise.csv` s environmentálními údaji vybraných evropských států.

2 Přepsání hodnot v řádku pomocí Update kurzoru

Pokud chceme obsah tabulky nejen číst, ale i měnit, použijeme **Update kurzor**. Syntaxe je prakticky stejná jako u `Search` kurzoru:

```
tab = arcpy.da.UpdateCursor("Sidla_body.shp", ["NAZEV", "KATEGORIE"])
```

Rozdíl je v tom, jaké metody má příslušný kurzor v nabídce. U `UpdateCursor` jde především o metodu `updateRow()`, pomocí které lze daný řádek, na kterém zrovna kurzor je, nahradit

jiným řádkem. Ten je třeba metodě předat v podobě n -tice nebo seznamu odpovídající délky, přičemž jednotlivé položky musejí svým datovým typem odpovídat otevřeným sloupcům.

Typy kurzorů - připomenutí

Kurzory v `arcpy` jsou trojího druhu:

- **SearchCursor** - pouze čtení hodnot z tabulky (minulá lekce)
- **UpdateCursor** - čtení a zápis do existujících řádků (tato lekce)
- **InsertCursor** - vkládání nových řádků do tabulky (tato lekce)

V následující ukázce změníme kategorii města Brno z 2 na 3 (tím o tomto městu nenaznačujeme nic špatného!):

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."
tab = arcpy.da.UpdateCursor("Sidla_body.shp", ["NAZEV", "KATEGORIE"],
                             '"NAZEV" = \'Brno\'' )
r = next(tab)
tab.updateRow(("Brno", 3))
del(tab)
```

Všimněte si, že jsme otevřeli rovnou pouze řádek s Brnem pomocí SQL dotazu. Přesto jsme museli použít funkci `next()`, abychom se na tento první řádek kurzorem posunuli.

2.1 Editace v cyklu

Častěji než úpravu jednoho řádku potřebujeme provést změny na více řádcích podle nějakého pravidla. V další ukázce provedeme přepisování řádku v rámci průchodu celé tabulky `for` cyklem. Do tabulky krajů přidáme textové pole “STAV” a vyplníme do něj hodnotu “OK” pro ty kraje, kde je zemřelých méně či stejně jako narozených, a “KO” pro ostatní. (Úlohu lze řešit i bez kurzorů. Jak byste to dělali?)

```
import arcpy

# Nastavení prostředí
arcpy.env.workspace = r"C:\my_path\..."
arcpy.env.overwriteOutput = True

# Vstupní data
input_fc = "Kraje.shp"
```

```
# Přidání nového pole "STAV"
arcpy.management.AddField(input_fc, "STAV", "TEXT", field_length=10)

# Vytvoření kurzoru pro aktualizaci dat
with arcpy.da.UpdateCursor(input_fc, ["NAROZENI", "ZEMRELI", "STAV"]) as cursor:

    # Procházení řádků a aktualizace pole "STAV"
    for row in cursor:
        if row[0] >= row[1]:
            row[2] = "OK"
        else:
            row[2] = "KO"

    # Aktualizace řádku v tabulce
    cursor.updateRow(row)
```

💡 Úprava řádku v cyklu

Při práci s UpdateCursor v cyklu:

1. Načteme řádek do proměnné (např. row)
2. Upravíme hodnoty v této proměnné (např. row[2] = "OK")
3. Zavoláme cursor.updateRow(row) pro uložení změn

Bez volání updateRow() se změny do tabulky neuloží!

Další příklad ukazuje, jak můžeme vypočítat nové hodnoty na základě existujících:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Přidání pole pro hustotu zalidnění
arcpy.management.AddField("Kraje.shp", "HUSTOTA", "DOUBLE")

# Výpočet hustoty pro každý kraj
with arcpy.da.UpdateCursor("Kraje.shp", ["POCET_OB", "SHAPE@AREA", "HUSTOTA"]) as cursor:
    for row in cursor:
        obyvatel = row[0]
        plocha_km2 = row[1] / 1000000 # převod z m² na km²

        # Výpočet hustoty
        row[2] = obyvatel / plocha_km2
```

```
cursor.updateRow(row)

print("Hustota zalidnění byla vypočítána.")
```

2.2 Úlohy

Úloha 16.1. Do tabulky krajů přidejte pole “VELIKOST” (TEXT) a vyplňte ho hodnotami: “velký” pro kraje s plochou nad 7000 km², “střední” pro plochu 4000-7000 km² a “malý” pro plochu pod 4000 km².

Úloha 16.2. Vytvořte nové pole “PODIL_ZEN” (DOUBLE) v tabulce krajů a vypočítejte do něj podíl žen k celkovému počtu obyvatel (v procentech).

Úloha 16.3. Pro všechna města kategorie 3 změňte kategorii na 4. Použijte SQL dotaz při otevírání kurzoru.

Úloha 16.4. Do tabulky krajů přidejte pole “PRUMERNY_VEK” a vypočítejte do něj průměrný věk obyvatel (použijte pole s počty obyvatel v různých věkových kategoriích, pokud jsou k dispozici, jinak zadejte fiktivní hodnoty).

3 Smazání řádku

Dalším použitím *Update* kurzoru je smazání řádku. Toho nyní využijeme a pokusíme se vymazat město Brno z mapy světa nadobro. Jelikož však následně budeme chtít svůj strašný čin odčinit a pomocí *Insert* kurzoru Brno opět na mapu světa vrátit, bude vhodné si příslušný záznam někam dopředu uložit, např. takto:

```
with arcpy.da.SearchCursor("Sidla_body.shp", ["SHAPE@", "NAZEV", "KATEGORIE"],
                           '"NAZEV" = \'Brno\''') as tab:
    brno = tab.next()
```

Pole SHAPE@

Pole **SHAPE@** obsahuje celý geometrický objekt prvku. Na rozdíl od **SHAPE@AREA** nebo **SHAPE@XY**, které vrací pouze konkrétní vlastnosti, **SHAPE@** vrací kompletní geometrii včetně všech souřadnic.

To je důležité při kopírování nebo přesouvání prvků - potřebujeme zachovat přesnou geometrii, ne jen její vlastnosti.

Nyní již samotné smazání:

```
with arcpy.da.UpdateCursor("Sidla_body.shp", ["SHAPE@", "NAZEV", "KATEGORIE"],
                           '"NAZEV" = \'Brno\''') as tab:
    next(tab)
    tab.deleteRow()
```

O výsledku se v tabulce měst přesvědčte sami v ArcGIS Pro...

3.1 Smazání více záznamů v cyklu

V další ukázce projdeme v cyklu všechny kraje, jejichž stav byl identifikován jako “KO”, a z mapy světa je jednoduše smažeme. (Raději si ovšem nejprve celou vrstvu zkopírujeme pro případ, že bychom toho v budoucnu litovali.)

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Vytvoření kopie dat
kraje_ok = arcpy.management.CopyFeatures("Kraje.shp", "Kraje_OK.shp")

# Smazání krajů se stavem "KO"
with arcpy.da.UpdateCursor(kraje_ok, ["STAV"]) as cursor:
    for row in cursor:
        if row[0] == 'KO':
            cursor.deleteRow()
```

Namísto podmínky `row[0] == 'KO'` můžeme filtraci “KO” krajů provést rovnou při otvírání kurzoru pomocí SQL dotazu:

```
with arcpy.da.UpdateCursor(kraje_ok, ["STAV"], where_clause='"STAV" = \'KO\''') as cursor:
    for row in cursor:
        cursor.deleteRow()
```

💡 SQL dotaz vs. podmínka v Pythonu

Oba přístupy fungují, ale SQL dotaz (`where_clause`) je efektivnější:

- Filtrování probíhá už při otvírání kurzoru
- Kurzor prochází méně řádků
- Vhodné zejména u velkých datasetů

Podmínku v Pythonu (`if row[0] == 'KO'`) použijte, když:

- Podmínka je složitější než lze vyjádřit v SQL
- Potřebujete provádět výpočty před rozhodnutím o smazání

3.2 Úlohy

Úloha 16.5. Smažte z tabulky měst všechna města kategorie 1. Nejprve si ale celý shapefile zkopírujte!

Úloha 16.6. Z tabulky krajů smažte všechny kraje, které mají hustotu zalidnění nižší než 80 obyvatel/km². (Pokud jste předtím nevytvořili pole HUSTOTA, vytvořte ho nyní.)

Úloha 16.7. Vytvořte skript, který projde všechny kraje a smaže ty, jejichž název začíná na písmeno “S” nebo “M”. Použijte SQL operátor LIKE.

Úloha 16.8. Vytvořte funkci `delete_by_attribute(fc, field, value)`, která smaže všechny záznamy, kde dané pole má zadanou hodnotu. Otestujte ji na libovolných datech.

4 Přidání nového řádku pomocí Insert kurzoru

Nyní, jak jsme avizovali, jsme si uvědomili dosah našeho brutálního činu - zrušení města Brna. Naštěstí je zde *Insert* kurzor, umožňující do tabulky vložit nový řádek. Má jednodušší syntax, neboť neumožňuje filtrování otvíraných řádků pomocí SQL dotazu. Jediné, co můžeme ovlivnit, je, které sloupce otevřeme. Nový řádek se přitom vždy přidává na konec tabulky (i v případě tak významného města, jakým je Brno).

Geometrie při vkládání prvků

Pokud pracujeme s atributovou tabulkou vektorové vrstvy, je nutné při přidávání nového záznamu vždy zadat i **geometrii prvku** do pole **SHAPE@**. Jinak by danému řádku neodpovídal žádný geometrický prvek a celá vrstva by byla poškozena. Pro běžné tabulky (`.dbf`, databázové tabulky) toto neplatí - tam geometrii nemáme.

Vrácení Brna do datasetu:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

with arcpy.da.InsertCursor("Sidla_body.shp", ["SHAPE@", "NAZEV", "KATEGORIE"]) as tab:
    tab.insertRow(brno)
```

Zbývá než doufat, že jste celou proceduru smazání a opětovné záchrany Brna dělali v rámci jedné session, tj. že jste mezitím nerestartovali konzoli. Proměnná `brno` by tak byla smazána a město nadobro ztraceno...

4.1 Vytvoření nové tabulky a její naplnění daty

Vkládání řádku si předvedeme na úloze, kdy vytvoříme úplně novou tabulku z environmentálních údajů vybraných evropských zemí:

Země	Emise CO (Mt)	Podíl OZE (%)	Podíl OZE v elektřině (%)	Změna emisí (2022-2023)	Hlavní OZE zdroje
Německo	572	22.0	52.0	-7.5%	Vítr, slunce, biomasa
Fran- cie	386	20.5	27.5	-7.5%	Vodní, vítr, slunce
Itálie	354	19.9	43.8	-7.5%	Vodní, slunce, vítr
Pol- sko	360	18.5	23.7	-7.5%	Vítr, biomasa
Španěl- sko	296	21.7	50.3	-7.5%	Vítr, slunce, vodní
Švéd- sko	43	66.4	87.5	-7.5%	Vodní, vítr, biomasa
Ni- zozem- sko	149	17.8	35.2	-7.5%	Vítr, slunce
Bel- gie	90	14.7	31.5	-7.5%	Vítr, slunce, biomasa
Česká re- pub- lika	89	18.1	15.9	-7.5%	Vítr, slunce, biomasa
Rak- ousko	61	36.5	87.8	-7.5%	Vodní, vítr, slunce
Fin- sko	46	50.8	47.4	-7.5%	Biomasa, vítr, vodní
Dán- sko	33	44.9	88.4	-7.5%	Vítr, biomasa, slunce
Norsko	37	79.4	97.8	-2.0%	Vodní, vítr

Zdroje dat:

- Emise CO : EDGAR (Emissions Database for Global Atmospheric Research) 2024, Energy Institute Statistical Review 2024
- Podíl OZE: Eurostat 2024, data za rok 2023
- Podíl OZE v elektřině: Eurostat 2024, data za rok 2024

Poznámky:

- Emise CO zahrnují pouze fosilní paliva a výrobu cementu
- OZE = Obnovitelné zdroje energie (podíl z celkové spotřeby energie)
- EU27 jako celek snížila emise o 7,5% mezi lety 2022-2023

4.1.1 Verze s daty v kódu

Nejprve verze, kdy data do kódu píšeme ručně (to je pracná alternativa, leč někdy se může hodit mít hodnoty napevno vepsané do kódu):

```
import arcpy

# Nastavení prostředí
arcpy.env.workspace = r"C:\my_path\..."
arcpy.env.overwriteOutput = True

# Vytvoření nové tabulky
output_table = "EU_OZE_Emise.dbf"
arcpy.management.CreateTable(arcpy.env.workspace, output_table)

# Přidání polí do tabulky
fields = [
    ("Zeme", "TEXT", 50),
    ("Emise_Mt", "DOUBLE"),
    ("Podil_OZE", "DOUBLE"),
    ("OZE_elekt", "DOUBLE"),
    ("Zmena_pct", "TEXT", 10),
    ("Zdroje", "TEXT", 100)
]

for field_name, field_type, *args in fields:
    if args: # Pokud je zadána délka
        arcpy.management.AddField(output_table, field_name, field_type,
                                   field_length=args[0])
    else:
        arcpy.management.AddField(output_table, field_name, field_type)
```

```
# Data k vložení
data = [
    ("Německo", 572, 22.0, 52.0, "-7.5%", "Vítr, slunce, biomasa"),
    ("Francie", 386, 20.5, 27.5, "-7.5%", "Vodní, vítr, slunce"),
    ("Itálie", 354, 19.9, 43.8, "-7.5%", "Vodní, slunce, vítr"),
    ("Polsko", 360, 18.5, 23.7, "-7.5%", "Vítr, biomasa"),
    ("Španělsko", 296, 21.7, 50.3, "-7.5%", "Vítr, slunce, vodní"),
    ("Švédsko", 43, 66.4, 87.5, "-7.5%", "Vodní, vítr, biomasa"),
    ("Nizozemsko", 149, 17.8, 35.2, "-7.5%", "Vítr, slunce"),
    ("Belgie", 90, 14.7, 31.5, "-7.5%", "Vítr, slunce, biomasa"),
    ("Česká republika", 89, 18.1, 15.9, "-7.5%", "Vítr, slunce, biomasa"),
    ("Rakousko", 61, 36.5, 87.8, "-7.5%", "Vodní, vítr, slunce"),
    ("Finsko", 46, 50.8, 47.4, "-7.5%", "Biomasa, vítr, vodní"),
    ("Dánsko", 33, 44.9, 88.4, "-7.5%", "Vítr, biomasa, slunce"),
    ("Norsko", 37, 79.4, 97.8, "-2.0%", "Vodní, vítr")
]

# Vložení dat do tabulky
field_names = [f[0] for f in fields]
with arcpy.da.InsertCursor(output_table, field_names) as cursor:
    for row in data:
        cursor.insertRow(row)
```

4.1.2 Import dat z CSV souboru

Nyní si předvedeme realističtější variantu - data máme uložené v CSV tabulce:

```
import arcpy
import csv
import os

# Nastavení prostředí
arcpy.env.workspace = r"C:\my_path\..."
arcpy.env.overwriteOutput = True

# Cesta k CSV souboru
csv_file = os.path.join(arcpy.env.workspace, "EU_OZE_Emise.csv")

# Vytvoření nové tabulky
output_table = "EU_OZE_Emise_from_CSV.dbf"
arcpy.management.CreateTable(arcpy.env.workspace, output_table)
```

```

# Přidání polí do tabulky
fields = [
    ("Zeme", "TEXT", 50),
    ("Emise_Mt", "DOUBLE"),
    ("Podil_OZE", "DOUBLE"),
    ("OZE_elekt", "DOUBLE"),
    ("Zmena_pct", "TEXT", 10),
    ("Zdroje", "TEXT", 100)
]

for field_name, field_type, *args in fields:
    if args:
        arcpy.management.AddField(output_table, field_name, field_type,
                                   field_length=args[0])
    else:
        arcpy.management.AddField(output_table, field_name, field_type)

# Načtení dat z CSV a vložení do tabulky
with open(csv_file, 'r', encoding='utf-8') as f:
    reader = csv.reader(f)
    next(reader) # Přeskočení hlavičky

    field_names = [f[0] for f in fields]
    with arcpy.da.InsertCursor(output_table, field_names) as cursor:
        for line in reader:
            # Převedení hodnot na správné datové typy
            row = [
                line[0],           # Zeme (TEXT)
                float(line[1]),    # Emise_Mt (DOUBLE)
                float(line[2]),    # Podil_OZE (DOUBLE)
                float(line[3]),    # OZE_elekt (DOUBLE)
                line[4],           # Zmena_pct (TEXT)
                line[5]            # Zdroje (TEXT)
            ]
            cursor.insertRow(row)

```

💡 Jednodušší alternativa - CopyRows

Uvedenou proceduru lze ve skutečnosti vyřešit jednodušeji bez kurzoru, pomocí nástroje `arcpy.management.CopyRows()`:

```
import arcpy

arcpy.env.workspace = r"C:\my_path\..."
arcpy.env.overwriteOutput = True

# Vstupní a výstupní tabulka
input_table = "EU_OZE_Emise.csv"
output_table = "EU_OZE_Emise_Copy.dbf"

# Kopírování tabulky
arcpy.management.CopyRows(input_table, output_table)
```

Tento nástroj automaticky rozpozná strukturu CSV a vytvoří odpovídající pole v DBF.

4.2 Úlohy

Úloha 16.9. Vytvořte novou tabulku “Mesice.dbf” s poli CISLO (SHORT) a NAZEV (TEXT) a naplňte ji názvy všech 12 měsíců v roce.

Úloha 16.10. Z existující tabulky krajů vytvořte novou tabulku obsahující pouze kraje s více než 500 000 obyvateli. Použijte `SearchCursor` pro čtení a `InsertCursor` pro vytvoření nové tabulky.

Úloha 16.11. Vytvořte CSV soubor s daty o pěti městech (název, počet obyvatel, rozloha) a importujte ho do nové DBF tabulky pomocí `InsertCursor`.

Úloha 16.12. Napište funkci `duplicate_features(input_fc, output_fc, where_clause)`, která zkopíruje vybrané prvky z jedné vrstvy do druhé včetně geometrie a všech atributů. Použijte `SearchCursor` a `InsertCursor`.

5 Cheatsheet

5.1 Základy Update Cursor

5.1.1 Vytvoření Update kurzoru

```
# Základní syntaxe
cursor = arcpy.da.UpdateCursor(in_table, field_names, where_clause)

# Příklad
```

```

cursor = arcpy.da.UpdateCursor("Kraje.shp", ["NAZEV", "POCET_OB"])

# S WHERE klauzulí
cursor = arcpy.da.UpdateCursor("Kraje.shp", ["NAZEV"], '"POCET_OB" > 1000000')

# S klauzulí with (doporučeno)
with arcpy.da.UpdateCursor("Kraje.shp", ["NAZEV", "POCET_OB"]) as cursor:
    for row in cursor:
        # práce s daty
        pass

```

5.1.2 Aktualizace řádku

```

# Aktualizace konkrétního řádku
with arcpy.da.UpdateCursor("Kraje.shp", ["NAZEV"], '"NAZEV" = \'Praha\') as cursor:
    row = next(cursor)
    cursor.updateRow(("Hlavní město Praha",))

# Aktualizace v cyklu
with arcpy.da.UpdateCursor("Kraje.shp", ["POCET_OB", "HUSTOTA"]) as cursor:
    for row in cursor:
        row[1] = row[0] / 1000 # výpočet nové hodnoty
        cursor.updateRow(row)

```

5.1.3 Podmíněná aktualizace

```

# S podmínkou v Pythonu
with arcpy.da.UpdateCursor("Kraje.shp", ["POCET_OB", "KATEGORIE"]) as cursor:
    for row in cursor:
        if row[0] > 1000000:
            row[1] = "velký"
        elif row[0] > 500000:
            row[1] = "střední"
        else:
            row[1] = "malý"
        cursor.updateRow(row)

# S WHERE klauzulí (efektivnější)

```

```

where = '"POCET_OB" > 1000000'
with arcpy.da.UpdateCursor("Kraje.shp", ["KATEGORIE"], where) as cursor:
    for row in cursor:
        row[0] = "velký"
        cursor.updateRow(row)

```

5.2 Mazání řádků

5.2.1 Smazání konkrétního řádku

```

# Smazání podle WHERE klauzule
with arcpy.da.UpdateCursor("Kraje.shp", ["NAZEV"], '"NAZEV" = \'Praha\''') as cursor:
    row = next(cursor)
    cursor.deleteRow()

```

5.2.2 Smazání více řádků

```

# Smazání všech řádků splňujících podmínku
with arcpy.da.UpdateCursor("Kraje.shp", ["POCET_OB"]) as cursor:
    for row in cursor:
        if row[0] < 100000:
            cursor.deleteRow()

# Efektivnější s WHERE klauzulí
where = '"POCET_OB" < 100000'
with arcpy.da.UpdateCursor("Kraje.shp", ["POCET_OB"], where) as cursor:
    for row in cursor:
        cursor.deleteRow()

```

5.2.3 Uložení před smazáním

```

# Uložení záznamu před smazáním
deleted_records = []

with arcpy.da.UpdateCursor("Kraje.shp", ["NAZEV", "POCET_OB"]) as cursor:
    for row in cursor:

```

```

        if row[1] < 100000:
            deleted_records.append(row)
            cursor.deleteRow()

print(f"Smazáno {len(deleted_records)} záznamů")

```

5.3 Insert Cursor

5.3.1 Vytvoření Insert kurzoru

```

# Základní syntaxe
cursor = arcpy.da.InsertCursor(in_table, field_names)

# Příklad
with arcpy.da.InsertCursor("Kraje.shp", ["NAZEV", "POCET_OB"]) as cursor:
    cursor.insertRow(("Nový kraj", 500000))

```

5.3.2 Vložení jednoho řádku

```

# Pro tabulku (bez geometrie)
with arcpy.da.InsertCursor("Tabulka.dbf", ["POLE1", "POLE2"]) as cursor:
    cursor.insertRow(("hodnota1", 123))

# Pro feature class (s geometrií)
with arcpy.da.InsertCursor("Body.shp", ["SHAPE@XY", "NAZEV"]) as cursor:
    cursor.insertRow([(x, y), "Nový bod"])

```

5.3.3 Vložení více řádků

```

# Z listu
data = [
    ("Kraj1", 100000),
    ("Kraj2", 200000),
    ("Kraj3", 150000)
]

```

```
with arcpy.da.InsertCursor("Kraje.shp", ["NAZEV", "POCET_OB"]) as cursor:
    for row in data:
        cursor.insertRow(row)
```

5.3.4 Import z CSV

```
import csv

with open("data.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)
    next(reader) # přeskočení hlavičky

    with arcpy.da.InsertCursor("Tabulka.dbf", ["POLE1", "POLE2"]) as cursor:
        for line in reader:
            # Převod datových typů
            row = (line[0], float(line[1]))
            cursor.insertRow(row)
```

5.4 Práce s geometrií

5.4.1 Kopírování prvku včetně geometrie

```
# Uložení prvku
with arcpy.da.SearchCursor("Body.shp", ["SHAPE@", "NAZEV"], '"NAZEV" = \'Praha\'') as cursor:
    saved_feature = next(cursor)

# Vložení na jiné místo
with arcpy.da.InsertCursor("Body_kopie.shp", ["SHAPE@", "NAZEV"]) as cursor:
    cursor.insertRow(saved_feature)
```

5.4.2 Vytvoření bodu na souřadnicích

```
# Pomocí SHAPE@XY
x, y = 500000, 6000000
with arcpy.da.InsertCursor("Body.shp", ["SHAPE@XY", "NAZEV"]) as cursor:
    cursor.insertRow([(x, y), "Nový bod"])
```

```
# Pomocí Point objektu
point = arcpy.Point(x, y)
with arcpy.da.InsertCursor("Body.shp", ["SHAPE@", "NAZEV"]) as cursor:
    cursor.insertRow([point, "Nový bod"])
```

5.5 Vytváření tabulek

5.5.1 Vytvoření prázdné tabulky

```
# Vytvoření tabulky
arcpy.management.CreateTable(workspace, "Nova_tabulka.dbf")

# Přidání polí
arcpy.management.AddField("Nova_tabulka.dbf", "NAZEV", "TEXT", field_length=50)
arcpy.management.AddField("Nova_tabulka.dbf", "HODNOTA", "DOUBLE")
arcpy.management.AddField("Nova_tabulka.dbf", "DATUM", "DATE")
```

5.5.2 Vytvoření a naplnění tabulky

```
# Vytvoření tabulky s poli
output = "Statistiky.dbf"
arcpy.management.CreateTable(arcpy.env.workspace, output)

fields = [
    ("Kategorie", "TEXT", 20),
    ("Pocet", "LONG"),
    ("Prumer", "DOUBLE")
]

for name, dtype, *length in fields:
    if length:
        arcpy.management.AddField(output, name, dtype, field_length=length[0])
    else:
        arcpy.management.AddField(output, name, dtype)

# Naplnění daty
data = [("A", 10, 5.5), ("B", 20, 7.3), ("C", 15, 6.1)]
```

```
with arcpy.da.InsertCursor(output, [f[0] for f in fields]) as cursor:
    for row in data:
        cursor.insertRow(row)
```

5.6 Kombinace kurzorů

5.6.1 Kopírování s úpravou

```
# Kopírování řádků s úpravou hodnot
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POCET_OB"]) as search:
    with arcpy.da.InsertCursor("Kraje_kopie.shp", ["NAZEV", "POCET_OB"]) as insert:
        for row in search:
            # Úprava hodnoty
            new_row = (row[0] + "_kopie", row[1] * 2)
            insert.insertRow(new_row)
```

5.6.2 Filtrované kopírování

```
# Kopírování pouze vybraných řádků
threshold = 500000

with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POCET_OB", "SHAPE@"]) as search:
    with arcpy.da.InsertCursor("Velke_kraje.shp", ["NAZEV", "POCET_OB", "SHAPE@"]) as insert:
        for row in search:
            if row[1] > threshold:
                insert.insertRow(row)
```

5.6.3 Agregace dat

```
# Sumarizace a uložení výsledků
from collections import defaultdict

statistiky = defaultdict(lambda: {"pocet": 0, "suma": 0})

# Čtení dat
with arcpy.da.SearchCursor("Okresy.shp", ["KRAJ", "POCET_OB"]) as cursor:
```

```

    for row in cursor:
        kraj = row[0]
        obyvatel = row[1]
        statistiky[kraj]["pocet"] += 1
        statistiky[kraj]["suma"] += obyvatel

# Vytvoření výstupní tabulky
output = "Statistiky_kraju.dbf"
arcpy.management.CreateTable(arcpy.env.workspace, output)
arcpy.management.AddField(output, "KRAJ", "TEXT", field_length=50)
arcpy.management.AddField(output, "POCET_OKRESU", "LONG")
arcpy.management.AddField(output, "CELKEM_OBYVATEL", "LONG")

# Zápis výsledků
with arcpy.da.InsertCursor(output, ["KRAJ", "POCET_OKRESU", "CELKEM_OBYVATEL"]) as cursor:
    for kraj, stats in statistiky.items():
        cursor.insertRow((kraj, stats["pocet"], stats["suma"]))

```

5.7 Časté vzory použití

5.7.1 Přidání vypočítaného pole

```

# Přidání pole a výpočet hodnot
arcpy.management.AddField("Kraje.shp", "HUSTOTA", "DOUBLE")

with arcpy.da.UpdateCursor("Kraje.shp", ["POCET_OB", "SHAPE@AREA", "HUSTOTA"]) as cursor:
    for row in cursor:
        row[2] = row[0] / (row[1] / 1000000) # obyvatel/km²
        cursor.updateRow(row)

```

5.7.2 Kategorizace hodnot

```

# Přidání kategorie podle hodnoty
arcpy.management.AddField("Kraje.shp", "VELIKOST", "TEXT", field_length=10)

with arcpy.da.UpdateCursor("Kraje.shp", ["POCET_OB", "VELIKOST"]) as cursor:
    for row in cursor:
        if row[0] > 1000000:

```

```

        row[1] = "Velký"
    elif row[0] > 500000:
        row[1] = "Střední"
    else:
        row[1] = "Malý"
    cursor.updateRow(row)

```

5.7.3 Normalizace textu

```

# Převod na jednotný formát
with arcpy.da.UpdateCursor("Kraje.shp", ["NAZEV"]) as cursor:
    for row in cursor:
        # Převod na velká písmena, odstranění mezer
        row[0] = row[0].upper().strip()
        cursor.updateRow(row)

```

5.8 Zpracování chyb

5.8.1 Try-except s kurzory

```

try:
    with arcpy.da.UpdateCursor("Kraje.shp", ["POCET_OB"]) as cursor:
        for row in cursor:
            row[0] = row[0] * 1.1
            cursor.updateRow(row)
        print("Aktualizace proběhla úspěšně")
except Exception as e:
    print(f"Chyba při aktualizaci: {e}")

```

5.8.2 Kontrola existence pole

```

# Kontrola před přidáním pole
field_name = "HUSTOTA"
existing_fields = [f.name for f in arcpy.ListFields("Kraje.shp")]

if field_name not in existing_fields:
    arcpy.management.AddField("Kraje.shp", field_name, "DOUBLE")

```

5.8.3 Ošetření None hodnot

```
with arcpy.da.UpdateCursor("Kraje.shp", ["POCET_OB", "HUSTOTA"]) as cursor:
    for row in cursor:
        if row[0] is not None:
            row[1] = row[0] / 1000
        else:
            row[1] = None
        cursor.updateRow(row)
```

5.9 Tipy pro výkon

5.9.1 Použití WHERE klauzule

```
# Místo podmínky v Pythonu (pomalé)
with arcpy.da.UpdateCursor("Kraje.shp", ["NAZEV"]) as cursor:
    for row in cursor:
        if row[0] == "Praha":
            # zpracování
            pass

# Použijte WHERE (rychlé)
with arcpy.da.UpdateCursor("Kraje.shp", ["NAZEV"], '"NAZEV" = \'Praha\'') as cursor:
    for row in cursor:
        # zpracování
        pass
```

5.9.2 Otevření pouze potřebných polí

```
# Špatně - otevření všech polí
with arcpy.da.UpdateCursor("Kraje.shp", ["*"]) as cursor:
    for row in cursor:
        row[5] = "nova_hodnota"
        cursor.updateRow(row)

# Dobře - pouze potřebná pole
with arcpy.da.UpdateCursor("Kraje.shp", ["NAZEV"]) as cursor:
```

```
for row in cursor:
    row[0] = "nova_hodnota"
    cursor.updateRow(row)
```

5.9.3 Batch operace

```
# Pro velké množství změn zvažte batch zpracování
batch_size = 1000
rows_to_update = []

with arcpy.da.UpdateCursor("Kraje.shp", ["NAZEV", "POCET_OB"]) as cursor:
    for row in cursor:
        row[1] = row[1] * 1.1
        rows_to_update.append(row)

    if len(rows_to_update) >= batch_size:
        # Zpracování batch
        for r in rows_to_update:
            cursor.updateRow(r)
        rows_to_update = []
```